arXiv:1805.00917v3 [stat.ML] 19 Nov 2018

A Scalable Discrete-Time Survival Model for Neural Networks

Michael F. Gensheimer[1] **and Balasubramanian Narasimhan**[2]

[1]**Department of Radiation Oncology, Stanford University School of Medicine**
[2]**Department of Statistics, Stanford University**

Corresponding author:
Michael F. Gensheimer[1]

Email address: mgens@stanford.edu

ABSTRACT

There is currently great interest in applying neural networks to prediction tasks in medicine. It is important for predictive models to be able to use survival data, where each patient has a known follow-up time and event/censoring indicator. This avoids information loss when training the model and enables generation of predicted survival curves. In this paper, we describe a discrete-time survival model that is designed to be used with neural networks, which we refer to as Nnet-survival. The model is trained with the maximum likelihood method using minibatch stochastic gradient descent (SGD). The use of SGD enables rapid convergence and application to large datasets that do not fit in memory. The model is flexible, so that the baseline hazard rate and the effect of the input data on hazard probability can vary with follow-up time. It has been implemented in the Keras deep learning framework, and source code for the model and several examples is available online. We demonstrate the performance of the model on both simulated and real data and compare it to existing models Cox-nnet and Deepsurv.

INTRODUCTION

With the popularization of deep learning and the increasing size of medical datasets, there has been increasing interest in the use of machine learning to improve medical care. Several recent papers have described use of neural network or other machine learning techniques to predict future clinical outcomes (Rajkomar et al., 2018; Kwong et al., 2017; Miotto et al., 2016; Avati et al., 2017). The outcome measure is generally evaluated at one follow-up time point, and there is often little discussion of how to deal with censored data (e.g., patients lost to follow-up before the follow-up time point). This is not ideal, as information about censored patients is lost and the model would need to be re-trained to make predictions at different time points. Because of these issues, modern predictive models generally use Cox proportional hazards regression or a parametric survival model instead of simpler methods such as logistic regression that discard time-to-event information (Cooney et al., 2009).

Several authors have described solutions for modeling time-to-event data with neural networks. These are generally adaptations of linear models such as the Cox proportional hazards model (Cox, 1972). Approaches include a discrete-time survival model with a heuristic loss function (Brown et al., 1997), a parametric model with predicted survival time having a Weibull distribution (Martinsson, 2016), and adaptations of the Cox proportional hazards model (Faraggi and Simon, 1995; Ching et al., 2018; Katzman et al., 2018). Most of the models assume proportional hazards (the effect of each predictor variable is the same at all values of follow-up time). This is not a very realistic assumption for most clinical situations. In the past, when models were typically trained using dozens or hundreds of patients, it was often not possible

to demonstrate violation of proportional hazards. However, in the modern era of datasets of thousands or millions of patients, it will usually be possible to demonstrate violation of the proportional hazards assumption, either by plotting residuals or with a statistical test.

In this paper, we describe Nnet-survival, a discrete-time survival model that is theoretically justified, naturally deals with non-proportional hazards, and is trained rapidly by minibatch gradient descent. It may be useful in several situations, especially when non-proportional hazards are known to be present, for very large datasets that do not fit in memory, or when predictor data is a good fit for a neural network approach (such as image or text data). We have published source code for the use of the model with the Keras deep learning library, which is available at `http://github.com/MGensheimer/nnet-survival`.

MATERIALS & METHODS

Relationship to prior work

In this section we describe prior approaches to the problem and illustrate some pitfalls that are addressed with our model.

Several authors have adapted the Cox proportional hazards model to neural networks (Faraggi and Simon, 1995; Ching et al., 2018; Katzman et al., 2018). This is potentially attractive since the Cox model has been shown to be very useful and is familiar to most medical researchers. One issue with this approach is that the partial likelihood for each individual depends not only on the model output for that individual, but also on the output for all individuals with longer survival. This would preclude the use of stochastic gradient descent (SGD) since with SGD only a small number of individuals are visible to the model at a time. Therefore the entire dataset would need to be used for each gradient descent step. This is undesirable because it slows down convergence, cannot be applied to datasets that do not fit into memory ("out-of-core learning"), and could result in getting stuck in a local minimum of the loss function (Bottou, 1991).

An alternative approach that avoids the above issue is to use a fully parametric survival model, such as a discrete time model. See section 7.5 of (Rodriguez, G., 2016) for a brief overview of discrete time survival models. Brown et al. proposed a discrete-time survival model using neural networks (Brown et al., 1997). This model can easily be trained with SGD, which is attractive. Follow-up time is divided into a set of fixed intervals. For each time interval the conditional hazard probability is estimated: the probability of failure in the interval, given that the individual has survived at least to the beginning of the interval. For each time interval j, the neural network loss is defined as (adapted from eq. 17 in (Brown et al., 1997)):

$$\frac{1}{2}\sum_{i=1}^{d_j}(1-h_j^i)^2+\frac{1}{2}\sum_{i=d_j+1}^{r_j}(h_j^i)^2 \tag{1}$$

where h_j^i is the hazard probability for individual i during time interval j, there are r_j individuals "in view" during the interval j (i.e., have not experienced failure or censoring before the beginning of the interval) and the first d_j of them suffer a failure during this interval. The overall loss function is the sum of the losses for each time interval.

The authors note that in the case of a null model with no predictor variables, minimizing the loss in (1) results in an estimate of the hazard probabilities that equals the Kaplan-Meier maximum likelihood estimate: $\hat{h_j}=\frac{d_j}{r_j}$. While this is true, the equivalence does not hold once each individual's hazard depends on the value of predictor variables.

A more theoretically justified loss function, which we use in our model, would be the negative of the log likelihood function of a statistical survival model. This likelihood function has been

well studied for discrete-time survival models in a non-deep learning context. Adapting eq. 3.4 from (Cox and Oakes, 1984) and eq. 2.17 from (Singer and Willett, 1993), the contribution of time interval j to the overall log likelihood is:

$$\sum_{i=1}^{d_j} \ln(h_j^i) + \sum_{i=d_j+1}^{r_j} \ln(1-h_j^i) \tag{2}$$

This is similar but not identical to (1) and can be shown to produce different values of the model parameters for anything more complex than the null model (for an example, see the file `brown1997_loss_function_example.md` in our GitHub repository).

The proposed model using (2) naturally incorporates time-varying baseline hazard rate and non-proportional hazards if each time interval output node is fully connected to the last hidden layer's neurons. The neural network has n-dimensional output where n is the number of time intervals, giving a separate hazard rate for each time interval.

There are several attractive features of the proposed model:

1. It is theoretically justified and fits into the established literature on survival modeling
2. The loss function depends only on the information contained in the current minibatch, which enables rapid training with minibatch SGD and application to arbitrary-size datasets
3. It is flexible and can be adapted to specific situations. For instance, for small sample size where we wish to minimize the number of neural network parameters, it is easy to incorporate a proportional hazards constraint so that the effect of the input data on the hazard function does not vary with follow-up time.

Model formulation

Follow-up time is divided into n intervals which are left-closed and right-open. Let $[t_1, t_2, ..., t_n]$ be the times at the upper limit of each interval. The conditional hazard probability h_j is defined as the probability of failure in interval j, given that the individual has survived at least to the beginning of the interval. h_j can vary per individual according to the input and the weights of the neural network. The predicted probability of an individual surviving at least to the end of interval j is:

$$S_j = \prod_{i=1}^{j} (1-h_i) \tag{3}$$

The model likelihood can be divided either by time interval as in (2), or by individual. For a neural network trained with minibatches of individuals, the latter formulation translates more easily into computer code. For an individual with failure during interval j (i.e., uncensored), the likelihood is the probability of surviving through intervals 1 through $j-1$, multiplied by the probability of failing during interval j:

$$lik = h_j \prod_{i=1}^{j-1} (1-h_i) \tag{4}$$

$$loglik = ln(h_j) + \sum_{i=1}^{j-1} \ln(1-h_i) \tag{5}$$

For a censored individual with a censoring time t_c which falls in the second half of interval $j-1$ or the first half of interval j (i.e., $\frac{1}{2}(t_{j-2}+t_{j-1}) \leq t_c < \frac{1}{2}(t_{j-1}+t_j)$), the likelihood is the probability of surviving through intervals 1 through $j-1$:

$$lik = \prod_{i=1}^{j-1}(1-h_i) \tag{6}$$

$$loglik = \sum_{i=1}^{j-1}\ln(1-h_i) \tag{7}$$

It can be seen that individuals with a censoring time in the second half of an interval are given "credit" for surviving that interval (without this, there would be a downward bias on the survival estimates (Brown et al., 1997).

The full log likelihood of the observed data is the sum of the log likelihoods for each individual. In the neural network survival model, we wish to maximize the likelihood, so we set the loss to equal the negative log likelihood and minimize the loss by by stochastic gradient descent or minibatch gradient descent.

Determination of hazard probability

For each time interval, the hazard probability will vary according to the input data. We have implemented two approaches to mapping input data to hazard probabilities:

With the **flexible** version, the final hidden layer (e.g., the "Max pooling" layer in Fig. 1) is densely connected to the output layer (the "Fully connected" layer in Fig. 1)). The output layer has n neurons, where n is the number of time intervals. The log odds of surviving each time interval is equal to the dot product of the incoming values and the kernel weights, plus the bias weight. Then, using a sigmoid activation function, log odds are converted to the conditional probability of surviving this interval. With this approach, both the baseline hazard rate and the effect of input data on the hazard rate can vary freely with follow-up time. This approach is most appropriate for larger datasets or when the proportional hazards assumption is known to be violated.

With the **proportional hazards** version, the baseline hazard probability is allowed to vary freely with time interval, but the effect of input data on hazard rate does not vary with follow-up time (if a certain combination of input data results in a high rate of death in the early follow-up period, it will also result in a high rate of death in the late follow-up period). This is implemented by setting the final hidden layer to have a single neuron, and densely connecting the prior hidden layer to the final hidden layer without any bias weights. The final hidden layer neuron value is $X\beta$, where X is the value of the prior hidden layer neurons and β is the weights between the prior hidden layer and the final hidden layer. The $X\beta$ notation is meant to echo that of the "linear predictor" in standard survival analysis, for instance section 18.2 of (Harrell Jr., 2015). The conditional probability of surviving the interval i is (adapted from eq. 18.13 in (Harrell Jr., 2015)):

$$1-h_i = (1-h_{base})^{exp(X\beta)} \tag{8}$$

where h_{base} is the baseline hazard probability for this time interval. The h_{base} values are estimated as part of the neural network by training a set of n weights, which are each transformed by a sigmoid function to convert baseline log odds of surviving each time interval into baseline